# Types of Variables and Storage Classes

Understanding variable types and storage classes in C programming

# Variable Types in C

## Global vs. Local variables

Global variables are recognized throughout the program whereas local variables are recognized only within the function where they are defined.

## Static vs. Dynamic variables

Retention of value by a local variable means, that in static, retention of the variable value is lost once the function is completely executed whereas in certain conditions the value of the variable has to be retained from the earlier execution and the execution retained.

The variables can be characterized by their **data type** and by their **storage class** .

# Data Type vs Storage Class

## Data Type

Refers to the type of value represented by a variable

**int, float, char, etc.**

## Storage Class

Refers to the permanence of a variable and its scope within the program

**auto, extern, static, register**

Scope = portion of the program over which variable is recognized

# Storage Classes

There are four different storage classes specified in C:

## Auto (matic)

Default storage class for local variables

## Extern (al)

Global variables visible to all files

## Static

Preserves value between function calls

## Register

Stored in CPU registers for fast access

The storage class associated with a variable can sometimes be established by the location of the variable declaration within the program or by prefixing keywords to variables declarations.

# What Storage Class Tells Us

- Where the variable would be stored
- What will be the initial value of the variable, if the initial value is not specifically assigned (i.e. the default initial value)
- What is the scope of the variable, i.e., in which functions the value of the variable would be available
- What is the life of the variables; i.e., how long would the variable exist

```
auto int a, b;
static int a, b;
extern float f;
```

# Automatic Variables

These variables come into existence whenever and wherever the variable is declared. These variables are also called as local variables, because these are available local to a function.

**Storage**

In the memory

**Default Value**

Garbage value

**Scope**

Local to the block in which it has been declared

**Life**

Till the control remains within the block

# Automatic Variables - Declaration

By default, a variable declared inside a function without storage class specification is an automatic.

```
// Both declarations are equivalent
int N;
or
auto int N;
```

## Example

```
main()
{
  auto int i=10;
```

```
  printf("%d",i);
}
```

The following Result: 10



# External Variables

These are not confined to a single function. Their scope ranges from the point of declaration to the entire remaining program. Therefore, their scope may be the entire program or two or more functions depending upon where they are declared.

## Points to remember

- These are global and can be accessed by any function within its scope

- Therefore value may be assigned in one and can be written in another
- There is difference in external variable definition and declaration
- External Definition is the same as any variable declaration
- Usually lies outside or before the function accessing it
- It allocates storage space required
- Initial values can be assigned
- The external specifier is not required in external variable definition



# External Variables - Declaration

```
extern int N;
```

## Important Points

- A declaration is required if the external variable definition comes after the function definition
- A declaration begins with an external specifier
- Only when external variable is defined is the storage space allocated
- External variables can be assigned initial values as a part of variable definitions, but the values must be constants rather than expressions
- If initial value is not included then it is automatically assigned a value of zero



# External Variables - Examples

In the following example, Variable "i" is a global variable. If the global variable declared outside (before function definition), there is no need to use extern declaration in function that use global variable.

## Example 1

```
int i=10; // global variable
main()
{
  int i=2;
  printf("%d",i);
  display();
}
display();
{
  printf("\n%d",i);
}
```

The following Result:

2

10

# External Variables – Examples with extern

Whereas, if global variable declared outside (after function definition), it has to be extern declaration within function that use global variable.

## Example 2

```
main()
{
  int i=2;
  printf("%d",i);
  display();
}
display();
{
  extern int i;
  printf("\n%d",i);
}
int i=10; //global variable
```

The following Result:
2

```
10
```



# Static Variables

💾 **Storage**

In the memory

🔢 **Default Initial Value**

Zero

📍 **Scope**

Local to the block in which it has been defined

⏱️ **Life**

The value of the variable persists between different function calls. The value will not disappear once the function in which it has been declared becomes inactive. It is

unavailable only when you come out the program.

```
static int N;
```



# Static Variables - Example

## Example

```
void value()
{
  static int a=5;
  a=a+2;
  printf("\t%d",a);
}
void main()
{
  value();
  value();
```

```
  value();
  getch();
}
```

The output of the program is not 7 7 7 but it is `7 9 11`



# Register Variables

**Storage**

In the CPU registers

**Default Initial Value**

Garbage value

**Scope**

Local to the block in which the variable is defined

**Life**

Till the control remains in the block in which it is defined

A value stored in a CPU register can always be accessed faster than the one which is stored in memory. Therefore, if a variable is used at many places in a program it is better to declare its storage class as register. A good example of frequently used variables is loop counters.



# Register Variables - Declaration

```
register int N;
```

## Example

```c
main()
{
  register int i=10;
  printf("%d",i);
}
```

Output: 10

## Important Note

The keyword 'register' is only a request to the compiler to store the variable in CPU register. The compiler may ignore this request if there are no free registers available.